mbusgw-t2r

Руководство пользователя

Номер документа: DO-T2RUG-RU-01A
Дата публикации: Ноябрь 2026
Сведения о ревизии: Это новое руководство.
Операционная система: Linux (glibc), ядро 4.x и новее
Версия программы: mbusgw-t2r V.01-01

NaPi World & StarLet Squad collaboration

mbusgw-t2r — Руководство пользователя

Copyright © 2026 NaPi World & StarLet Squad collaboration

Правовая информация

Сведения в настоящем документе могут быть изменены без предварительного уведомления. NaPi World & StarLet Squad collaboration не несёт ответственности за технические или редакционные ошибки и пропуски в настоящем документе.

Программа, описанная в настоящем руководстве, распространяется по лицензии Apache License, Version 2.0; см. файлы LICENSE и NOTICE в составе дистрибутива.

Программа, описанная в настоящем руководстве, распространяется в надежде, что она будет полезной, но БЕЗ КАКИХ-ЛИБО ГАРАНТИЙ, в том числе без подразумеваемых гарантий товарной пригодности и пригодности для конкретной цели.

MODBUS — зарегистрированный товарный знак Schneider Electric USA, Inc. Linux — зарегистрированный товарный знак Линуса Торвальдса. UNIX — зарегистрированный товарный знак The Open Group.

Содержание

Предисловие iv

1. О настоящем руководстве iv

2. Для кого написано iv

3. Структура документа iv

4. Смежные документы iv

5. Соглашения iv

Глава 1. Что делает шлюз 1

Глава 2. Перед началом 2

Глава 3. Файл настроек 3

3.1. Раздел serials 3

3.2. Раздел listeners 4

3.3. Режим RAW (прозрачный) 4

Глава 4. Запуск шлюза 6

4.1. Первая проверка с mbpoll 6

4.2. На OpenWrt 7

4.3. Как смотреть за шлюзом и управлять им: mbus-ctl 7

Глава 5. Псевдоустройство меток времени 9

Глава 6. Как читать журнал 10

Глава 7. Поиск неисправностей 11

7.1. Симптом, причина, действие 11

7.2. Справочник кодов сообщений 13

7.3. Если ничего не помогло 17

Предисловие

1. О настоящем руководстве

Настоящее руководство описывает настройку, эксплуатацию и поиск неисправностей mbusgw-t2r — шлюза, который переводит запросы MODBUS TCP, приходящие из сети, в транзакции MODBUS RTU на последовательной линии (RS-485 или RS-232) и возвращает ответы обратно.

2. Для кого написано

Руководство рассчитано на читателя без опыта работы с MODBUS и администрирования Linux: каждый шаг расписан, ничего не подразумевается. Опытный инженер может сразу перейти к главе 3 (файл настроек) и главе 7 (поиск неисправностей).

3. Структура документа

Глава 1 объясняет, что делает шлюз. Глава 2 — контрольный список перед запуском. Глава 3 описывает файл настроек ключ за ключом. Глава 4 — запуск, опции командной строки и первая сквозная проверка. Глава 5 — встроенное псевдоустройство меток времени. Глава 6 учит читать журнал. Глава 7 — справочник по поиску неисправностей: симптом, причина, действие, и справочник всех кодов сообщений.

4. Смежные документы

README_RU.md — последовательность установки (пакет StarLet, сборка, `make install`) и автоматическая сборка в GitLab CI (`.gitlab-ci.yml`). MODBUS over Serial Line Specification and Implementation Guide и MODBUS Application Protocol Specification — сам протокол, доступны на modbus.org.

5. Соглашения

Обозначение	Значение
`Моноширинный`	Примеры кода, имена файлов, команды и экранный вывод.
`%T2R-E-КОД`	Код сообщения, как он выглядит в журнале шлюза; см. раздел 7.2.
`<...>`	Место подстановки фактического значения, например имени устройства.
`$`	Приглашение оболочки непривилегированного пользователя; вводить его не нужно.
`Ctrl/C`	Удерживая клавишу Ctrl, нажмите клавишу C.

Глава 1. Что делает шлюз

У вас есть устройство (счётчик электроэнергии, терморегулятор, ПЛК …), которое говорит на MODBUS RTU по последовательной линии. И есть программа (SCADA, скрипт, `mbpoll` …), которая говорит на MODBUS TCP по сети. Шлюз стоит между ними и переводит, как показано на рисунке 1.

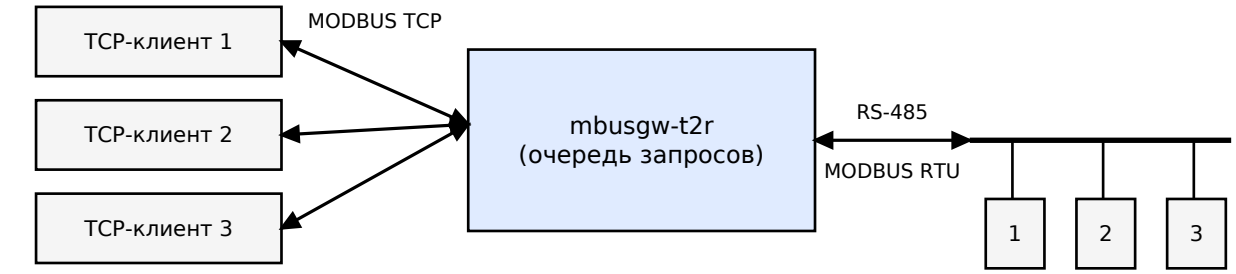


Рисунок 1. Путь данных через шлюз

Несколько TCP-клиентов могут подключаться одновременно: шлюз выстраивает их в очередь, чтобы на линию уходил только один запрос за раз — линия RTU физически не умеет нести две транзакции сразу.

На одной линии RS-485 может сидеть несколько устройств. Они различаются адресом ведомого внутри самого MODBUS-запроса; шлюзу их адреса знать не нужно вовсе.

Глава 2. Перед началом

Пройдите этот список один раз — он сэкономит час гаданий потом.

1. Шлюз установлен (см. README_RU.md, раздел «Установка»). После установки у вас есть программа `/opt/mbusgw-t2r/sbin/mbusgw-t2r` и файл настроек `/opt/mbusgw-t2r/etc/modbus-t2r_settings.conf`.

2. Вы знаете, к какому последовательному порту подключено устройство. В Linux это файл вида `/dev/ttyS0` или `/dev/ttyUSB0`. Если не уверены — выдерните и вставьте USB-адаптер и выполните:

```
$ dmesg | tail
```

3. Вы знаете параметры линии устройства: скорость, биты данных, чётность, стоп-биты. Они написаны в паспорте устройства; самый распространённый набор — `9600, 8, N, 1`.

4. Ваш пользователь имеет право открыть порт. Проверьте группу файла порта:

```
$ ls -l /dev/ttyUSB0
crw-rw---- 1 root dialout 188, 0 ... /dev/ttyUSB0
```

Если группа `dialout` — добавьте себя в неё и перезайдите в систему (либо просто запускайте шлюз через `sudo`):

```
$ sudo usermod -a -G dialout $USER
```

Глава 3. Файл настроек

В файле два раздела: `serials` (последовательные линии) и `listeners` (TCP-порты). Минимальный рабочий пример:

```
serials = (
{ device = "/dev/ttyUSB0";
  chars  = "9600, 8, N, 1";
  iotmo  = 1000;
}
);

listeners = (
{ bind   = "TCP:0.0.0.0:502";
  target = "/dev/ttyUSB0";
  connlm = 8;
}
);
```

Читается так: «открой порт /dev/ttyUSB0 на 9600-8-N-1 и жди ответа до 1000 мс; слушай TCP-клиентов на всех интерфейсах, порт 502, и пересылай их на этот порт».

3.1. Раздел serials

По одной записи на каждый последовательный порт. Таблица 3-1 перечисляет все ключи.

Ключ	Обяз.	Значение и допустимые величины
`device`	да	Файл последовательного порта, например /dev/ttyUSB0.
`chars`	да	Параметры линии: скорость, биты данных, чётность, стоп-биты. Скорость 50..4000000; биты данных 5..8; чётность N (нет), E (чётная), O (нечётная); стоп-биты 1..2.
`iotmo`	нет	Сколько ждать ответа устройства, миллисекунды. По умолчанию 1000. Берите с запасом относительно самого медленного устройства на линии: опоздавший ответ теряется.
`lategd`	нет	После неудачного запроса (нет ответа, битый ответ) ещё столько миллисекунд слушать линию и выбрасывать всё, что пришло (LATEANS): опоздавший ответ не должен уйти следующему запросу. Время тратится только после сбоя: запрос к неработающему устройству занимает iotmo + lategd. 0..60000, по умолчанию iotmo / 4; 0 — только пауза в 3,5 символа, как до X.00-13ECO08.
`rs485`	нет	1 — просить ядро управлять направлением RS-485 (только для портов, которые это умеют). По умолчанию 0.
`ts_enabled`	нет	1 — включить псевдоустройство меток времени (глава 5). По умолчанию 0.
`ts_unit`	нет	Адрес ведомого, на который отвечает псевдоустройство. По умолчанию 135.
`ts_fncode`	нет	Код функции, на который оно отвечает. По умолчанию 4.
`ts_base_reg0`	нет	Первый регистр метки времени. По умолчанию 135.

Если запись неправильная, шлюз пропускает её и говорит почему — с допустимым диапазоном прямо в сообщении:

```
%T2R-E-SERRANGE, [serial #00:</dev/ttyUSB0>] --- speed (baud) 31 is out of range [50..4000000]
```

USB-адаптеры: переподключать можно. С X.00-13ECO09 шлюз замечает, что адаптер пропал (DEVLOST) — даже когда никто не опрашивает устройство — и сразу закрывает порт. Раз в секунду он пытается открыть порт снова по имени; пока адаптера нет, клиенты получают исключение 10 (GATEWAY_PATH_UNAVAILABLE), а не 4. Когда адаптер вернулся, в журнале DEVREADY и работа продолжается, перезапуск не нужен. Если адаптера нет при старте, TCP-порт всё равно открывается (DEVABSENT).

Используйте постоянное имя. Ядро раздаёт номера ttyUSB0, ttyUSB1, ... в порядке появления адаптеров, и после переподключения тот же адаптер может получить другой номер. Пишите в device (и в target листенера) постоянное имя: имя по USB-разъёму из ls -l /dev/serial/by-path/ (порты 4-портового FT4232H отличаются частью :1.0 ... :1.3) или своё имя через правило udev:

```
SUBSYSTEM=="tty", ENV{ID_PATH}=="platform-xhci-hcd.0-usb-0:1.1:1.0", SYMLINK+="rs485-1"
```

в /etc/udev/rules.d/99-rs485.rules даёт /dev/rs485-1 (значения смотрите в udevadm info /dev/ttyUSB0).

3.2. Раздел listeners

Ключ	Обяз.	Значение и допустимые величины
`bind`	да	Где слушать: TCP:<IP-адрес>:<порт>. Порт 1..65535; 0.0.0.0 — все интерфейсы. UDP не поддерживается.
`target`	да	На какой порт пересылать. Должен дословно, символ в символ, совпадать с device из serials.
`connlm`	нет	Сколько TCP-клиентов может быть подключено к этому порту одновременно; лишний сразу отключается (CONNLIMIT). По умолчанию 32, диапазон 1..512.
`idletmo`	нет	Клиент, который ничего не присылает столько секунд, отключается (IDLETMO). По умолчанию 60, диапазон 1..86400.
`mode`	нет	Что делает слушатель: "MODBUS" (по умолчанию) — шлюз MODBUS TCP в RTU, "RAW" — прозрачный поток октетов (см. 3.3). Заглавные или строчные буквы — не важно; любое другое слово — ошибка (LSNBADMODE).
`takeover`	нет	Только для RAW. 1 (по умолчанию): новый клиент вытесняет текущего (RAWKICK). 0: пока подключён другой клиент, новый отклоняется (RAWBUSY). В режиме MODBUS этот ключ — ошибка (LSNRAWOPT).
`iotmo`	нет	Зарезервирован, пока не используется.

Порт 502 — стандартный порт MODBUS TCP, но в Linux порты ниже 1024 требуют прав root. Не хотите работать под root — возьмите, например, `5502` и укажите его в клиенте. Несколько MODBUS-слушателей могут разделять один target: шлюз выстроит их запросы в очередь. В режиме RAW два умолчания другие: connlm может быть только 1, а idletmo равен 86400 (не 60). RAW-слушателю последовательный порт нужен целиком.

3.3. Режим RAW (прозрачный)

В режиме MODBUS шлюз понимает MODBUS. В режиме RAW — нет. Он просто переносит октеты от TCP-клиента в последовательную линию и из линии клиенту. Ничего не меняет. Это «виртуальный COM-порт по сети».

Типичный случай: на ПК с Windows есть программа для старого прибора. Программе нужен COM-порт. Вы ставите на ПК программу «COM через TCP» (она создаёт виртуальный COM-порт и соединяет его со шлюзом по TCP). Программа думает, что работает с настоящим COM-портом. Прибор может говорить на любом последовательном протоколе, не только на MODBUS.

```
[Программа на ПК] --- виртуальный COM --- TCP --->
        [mbusgw-t2r, режим RAW] --- линия ---> [Прибор]
```

Пример настроек:

```
listeners = (
    { bind = "TCP:0.0.0.0:4001"; mode = "RAW"; target = "/dev/ttyUSB1"; }
);
serials = (
    { device = "/dev/ttyUSB1"; chars = "19200, 8, E, 1"; flow = "NONE"; iotmo = 1000; }
);
```

Правила режима RAW:

• Один клиент за раз. connlm может быть только 1.

• Новый клиент вытесняет старого (takeover = 1, по умолчанию). Старое соединение закрывается, линию получает новый; в журнале будет RAWKICK. При takeover = 0 старый клиент остаётся, а новому отказывают (RAWBUSY).

• Последовательный порт — только для этого слушателя. Ни один другой слушатель, MODBUS или RAW, не может брать тот же target (LSNRAWSHR; порядок записей не важен). MODBUS-слушатели по-прежнему могут делить порт между собой.

• Скорость и чётность берутся из шлюза, из записи serials. Если программа на ПК ставит у своего виртуального COM-порта другую скорость, в линию это не передаётся. Задайте верный chars в записи serials.

• Долгое молчание — это нормально. idletmo по умолчанию — 86400 секунд (сутки). Мёртвого клиента (ПК упал, кабель оборван) находит TCP keepalive, и его отбрасывают примерно за 1 минуту (30 секунд тишины, потом 3 пробы с шагом 10 секунд).

• Меток времени нет. Псевдоустройство ts_* (глава 5) в режиме RAW не работает. В начале сессии старые октеты в буферах последовательного порта выбрасываются.

• Нет последовательного порта — нет сеанса. Если USB-адаптера нет или его выдернули (DEVABSENT, DEVLOST, 3.1), новый клиент сразу отключается, без данных, а идущий сеанс закрывается. Когда адаптер вернётся, клиент просто подключается снова.

Программа в Linux как клиент. Драйвер псевдопоследовательного порта serial2net (NaPi Lab, X.00-08ECO1 и новее) создаёт /dev/ser2net00, /dev/ser2net01 и так далее и переносит всё, что туда записано, как есть на TCP-партнёра. Направьте партнёра на RAW-слушатель (p_ser2net_peer="127.0.0.1:4001"), и любая программа, которой нужен последовательный порт (mbpoll -m rtu, minicom, pyserial), работает с удалённой линией как с местным портом. Скорость по-прежнему берётся из записи serials шлюза. Проверено: 1 МиБ в каждую сторону блоками с CRC32 (без потерь), четыре порта одновременно с mbpoll, 115200.

Замечание для обоих режимов: начиная с X.00-14 вход последовательного порта «бинарно чистый». Порт больше не превращает CR в LF и не обрезает 8-й бит. Раньше октет 0x0D мог прийти как 0x0A на порту с настройками по умолчанию.

Глава 4. Запуск шлюза

Вручную, в терминале (сообщения идут прямо на экран):

```
$ /opt/mbusgw-t2r/sbin/mbusgw-t2r /settings=/opt/mbusgw-t2r/etc/modbus-t2r_settings.conf
```

Опция	Смысл
`/settings=<файл>`	Файл настроек (глава 3).
`/trace`	Подробная трассировка: каждый запрос и ответ печатается в hex. Бесценно при первом запуске.
`/logfile=<файл>`	Писать журнал в файл вместо экрана.
`/logsize=<октет>`	Ротировать файл журнала при превышении размера.
`/control=<файл>`	Файл-сокет для mbus-ctl (раздел 4.3), NONE — без него.
`/check`	Только проверить файл настроек и выйти (раздел 4.3).

Здоровый запуск печатает (сокращённо):

```
%T2R-I-REVISNF, Rev: T2R X.00-06/aarch64(built at ...) (REV: 00.06.00)
%T2R-I-DEVADDED, Added device #00 [</dev/ttyUSB0>, Chars: <9600, 8, N, 1>, ...] --- added
%T2R-I-LSNADDED, Added listener #00 [Target: </dev/ttyUSB0>, Net: <TCP:0.0.0.0:502>, ...] --- added
%T2R-S-DEVREADY, Device </dev/ttyUSB0> [9600 baud, answer tmo: 1000 msec, t3.5: 4010 usec] --- is r
%T2R-S-LSNRRDY, [#3] Listener 0.0.0.0:502 [Target: </dev/ttyUSB0>] --- is ready
```

Важнее всего две строки: DEVREADY (порт открыт) и LSNRRDY (TCP-порт слушает). Видите обе — шлюз работает. Рисунок 2 показывает, что происходит дальше с каждым запросом.

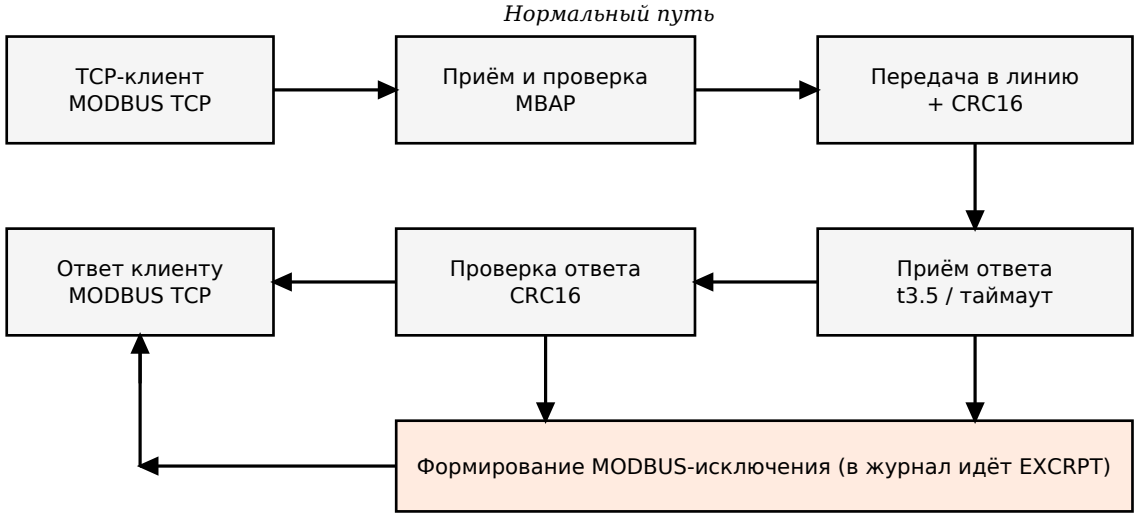


Рисунок 2. Жизненный цикл одного запроса

Остановка: один раз Ctrl/C и секунда ожидания. Переключить трассировку у уже работающего шлюза без перезапуска:

```
$ kill -USR1 <pid>
```

4.1. Первая проверка с mbpoll

`mbpoll` — маленький свободный MODBUS-клиент (`sudo apt install mbpoll`). Читаем 3 регистра хранения с устройства с адресом 1:

```
$ mbpoll -a 1 -r 1 -c 3 -1 127.0.0.1 -p 502
```

Если вернулись числа — всё работает, из конца в конец.

4.2. На OpenWrt

На роутер с OpenWrt ставьте пакет для своей платы со страницы Releases (архитектура — строка `DISTRIB_ARCH` в `/etc/openwrt_release`): `.ipk` для OpenWrt 24.10, `.apk` для OpenWrt 25.12. Пакету больше ничего не нужно, сеть роутеру не нужна.

```
# opkg install ./mbusgw-t2r_1.1.0-*_mipsel_24kc.ipk
# apk add --allow-untrusted ./mbusgw-t2r_1.1.0-*_mipsel_24kc.apk
```

Файлы лежат не там, где на Debian: программа — `/usr/sbin/mbusgw-t2r`, файл настроек — `/etc/mbusgw-t2r/modbus-t2r_settings.conf`. После установки служба выключена. Настройте файл настроек, затем включите службу и запустите её:

```
# uci set mbusgw-t2r.main.enabled=1 && uci commit mbusgw-t2r
# /etc/init.d/mbusgw-t2r start
# logread -e mbusgw-t2r
```

Последняя команда показывает сообщения шлюза (они идут в системный журнал). Адаптеру USB-serial нужен модуль ядра: `kmod-usb-serial-ftdi`, `kmod-usb-serial-ch341` или `kmod-usb-serial-cp210x` — по микросхеме адаптера.

4.3. Как смотреть за шлюзом и управлять им: mbus-ctl

`mbus-ctl` — маленькая программа рядом со шлюзом (`/opt/mbusgw-t2r/sbin/mbus-ctl`; пакеты .deb и OpenWrt кладут её в PATH как `/usr/sbin/mbus-ctl`). Она спрашивает работающий шлюз, что он делает, а ещё умеет отключить клиента или перезапустить шлюз. Говорит она со шлюзом через файл-сокет на том же компьютере, `/var/run/mbusgw-t2r.sock`. В сеть ничего не открывается. Пользоваться файлом могут только root и группа шлюза (права 0660), поэтому пишите перед командой `sudo`.

Команда	Что вы получите
`show connections`	Кто подключён к какому TCP-порту, с какого времени, запросы, ответы, ошибки и октеты.
`show connections /continuous`	То же, с перерисовкой каждые 2 секунды до Ctrl/C (/continuous=5 — каждые 5 секунд).
`show lines`	Последовательные порты: устройство, скорость, таймауты, OPEN или ABSENT, сколько клиентов.
`show gateway`	Версия, PID, сколько работает, файл настроек, TCP-порты.
`reset connection 7`	Отключить клиента с Id 7 (из show connections).
`reset connection 10.0.0.7`	Отключить всех клиентов с этого адреса (10.0.0.7:44770 — только одного).
`reset connection /all`	Отключить всех клиентов.
`restart gateway`	Проверить файл настроек и перезапустить шлюз с ним.

Слова можно сокращать: `sho conn /cont`, `sho gw`, `reset conn 7`, `restart gw`. SERIALS — то же, что LINES, GW — то же, что GATEWAY. Слишком коротко только `res`: это и RESET, и RESTART.

```
$ sudo mbus-ctl sho conn
Connections of the MODBUS gateway at 07-NOV-2026 00:07:44
```

```
Listener #0  0.0.0.0:502  MODBUS -> /dev/ttyUSB0  UP, 2 of 8 connection(s)
    Id  Sd  Client            Connected             Requests  Answers  Errors
     1   8  10.0.0.7:44770    07-NOV-2026 00:07:44         7        7       0
     2   9  10.0.0.9:44772    07-NOV-2026 00:07:44         3        3       0

Total: 2 connection(s)
```

Id — номер соединения для reset. Sd — число в [#8] в строках журнала этого клиента. Errors — ответы с исключением MODBUS плюс запросы, на которых соединение оборвалось.

restart gateway сначала проверяет файл настроек в отдельном процессе (`mbusgw-t2r /settings=<файл> /check`). Если в файле ошибка, ничего не перезапускается: вы видите строку ошибки, шлюз работает дальше со старыми настройками. Если файл хороший, шлюз отключает клиентов, закрывает порты и запускает сам себя заново с той же командной строкой. PID остаётся прежним: systemd и procd ничего не замечают. mbus-ctl ждёт, пока шлюз снова ответит (CTLBACK), 3–15 секунд. Под systemd то же делает `systemctl reload mbusgw-t2r`. После обновления пакета используйте `systemctl restart`.

Шлюз понимает обычные текстовые строки, поэтому его можно спросить и руками:

```
$ echo "SHOW CONNECTIONS" | sudo socat - UNIX-CONNECT:/var/run/mbusgw-t2r.sock
```

Двум шлюзам на одном компьютере нужны два файла-сокета: `/control=<файл>` для шлюза, `/socket=<файл>` для mbus-ctl.

Глава 5. Псевдоустройство меток времени

Шлюз умеет отвечать сам — не трогая линию — на запрос чтения, посланный на специальный адрес ведомого. Ответ — текущее время машины, на которой работает шлюз. Так SCADA может читать точное время по обычному MODBUS.

При `ts_enabled = 1; ts_unit = 135; ts_fncode = 4; ts_base_reg0 = 135;` запрос «прочитай N входных регистров начиная со 135-го у ведомого 135» возвращает регистры рисунка 3.

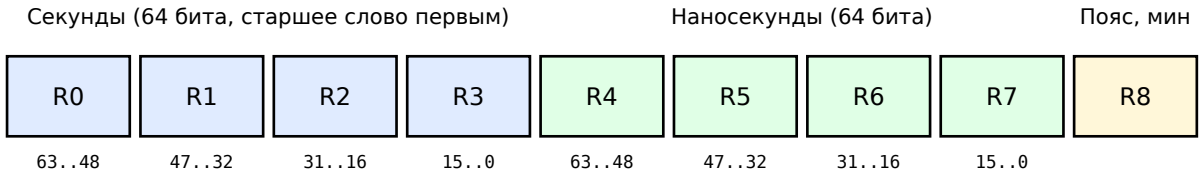


Рисунок 3. Раскладка регистров метки времени

Запрошено регистров	Что получаете
4	Секунды эпохи UNIX, 64 бита, старшее 16-битное слово первым.
8	Плюс наносекунды, упакованные так же в R4..R7.
9	Плюс R8 — смещение местного часового пояса, в знаковых минутах (Москва = +180).

Любое другое число регистров в TS-запросе получает MODBUS-исключение ILLEGAL_DATA_ADDRESS — это сделано намеренно.

Глава 6. Как читать журнал

Каждое важное событие — одна строка с кодом:

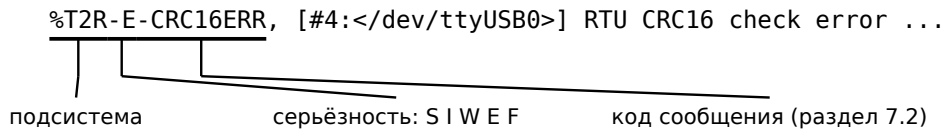


Рисунок 4. Анатомия сообщения журнала

Буква серьёзности: S — успех, I — информация, W — предупреждение, E — ошибка, F — фатально. Часть `[#4:</dev/ttyUSB0>]` означает: файловый дескриптор #4, вот этот последовательный порт.

Ищите в журнале по коду — именно для этого коды и существуют:

```
$ grep CRC16ERR /var/log/mbusgw-t2r.log
```

Глава 7. Поиск неисправностей

Золотое правило: запустите с /trace и читайте журнал. Шлюз всегда говорит, что именно ему не нравится.

7.1. Симптом, причина, действие

Вы видите	Что это значит / что делать
`%T2R-E-DEVOPNERR, ... errno: 2`	Файла порта нет: адаптер выдернут или имя неверное. Выполните dmesg | tail после втыкания; поправьте device в настройках. С X.00-13ECO09 TCP-порт всё равно открывается (DEVABSENT), устройство пробуется раз в секунду.
`%T2R-E-DEVLOST, ...`	USB-адаптер пропал (выдернут, сброс хаба, просадка питания); клиенты получают исключение 10. Если вернётся — ничего делать не нужно, следом будет DEVREADY. Если нет: dmesg | tail, кабель, питание; если вернулся под другим номером ttyUSB — постоянное имя (3.1).
`%T2R-W-DEVABSENT, ...`	При старте последовательный порт не открылся (почему — в строке выше); его TCP-порт работает и отвечает исключением 10. Подключите адаптер или поправьте device.
`Клиент получает исключение 0x0A (10)`	Последовательного порта этого TCP-порта нет или он пропал: см. DEVLOST, DEVABSENT.
`%T2R-E-DEVOPNERR, ... errno: 13`	Нет прав доступа. Запустите под sudo или добавьте себя в группу dialout.
`%T2R-E-DEVOPNERR, ... errno: 16`	Порт занят. Найдите держателя: sudo fuser /dev/ttyUSB0, и остановите его.
`%T2R-E-NOANSWER, ...`	Запрос ушёл, ответа нет: 1) неверный адрес ведомого в запросе клиента; 2) провода (поменяйте A/B у RS-485); 3) не та скорость/чётность — перепроверьте chars; 4) устройство выключено.
`%T2R-E-CRC16ERR, ...`	Байты приходят повреждёнными: 1) chars не совпадает с устройством (самое частое!); 2) помехи — земля, терминаторы, длина кабеля; 3) два мастера на одной линии.
`CRC16ERR на части запросов после перезагрузки или переподключения`	Шлюз старше X.00-13ECO07 оставлял у свежего порта замену CR на NL (icrnl): байт 0x0D становился 0x0A. Обновитесь; для старого шлюза: ExecStartPre=/bin/stty -F /dev/ttyUSB0 raw (на каждый порт) в юните.
`%T2R-E-BADFRAME, ...`	Что-то пришло, но это не осмысленный MODBUS-кадр: обычно те же причины, что у CRC16ERR; 400+ октетов мусора — на линии чужое, не-MODBUS устройство.
`CRC16ERR / BADFRAME время от времени, только на USB-адаптере FTDI`	Адаптер отдаёт ответ кусками, раз в 16 мс (latency_timer). С X.00-10 шлюз дожидается всего ответа для функций 01-06, 15, 16, 23; остальные по-прежнему обрезаются. Поставьте latency_timer в 1 мс (ответы заодно станут быстрее): скопируйте /opt/mbusgw-t2r/share/doc/mbusgw-t2r/99-ftdi-latency.rules в /etc/udev/rules.d/, выполните sudo udevadm control --reload && sudo udevadm trigger --subsystem-match=usb-serial; cat /sys/bus/usb-serial/devices/ttyUSB0/latency_timer должна напечатать 1.

Вы видите	Что это значит / что делать
`%T2R-W-FRAMETMO, ...`	Ответ начался, но не завершился: очень медленное или зависшее устройство. Увеличьте iotmo; проверьте кабель.
`%T2R-W-LATEANS, ... после NOANSWER`	Устройство ответило, но позже iotmo; ответ выброшен. Увеличьте iotmo с запасом относительно самого медленного устройства на линии.
`%T2R-W-ECHOMISM, ...`	Ответ на запись (функции 5, 6, 15, 16) — не эхо этого запроса: обычно опоздавший ответ на предыдущий (увеличьте iotmo); если устройство быстрое — два мастера на линии; если на каждой записи в одно устройство — оно отвечает не эхом, как требует спецификация: запись выполнена, но считается неудачной, сообщите нам модель.
`%T2R-W-EXCRPT, ... (SERVER_DEVICE_FAILURE)`	Шлюз сообщил клиенту «на последовательной стороне сбой». Настоящая причина — в предыдущей строке журнала (NOANSWER, CRC16ERR ...).
`%T2R-W-CONNLIMIT, ...`	К порту уже подключено connlm клиентов. Если все настоящие — увеличьте connlm этого слушателя; иначе найдите, кто держит соединения: ss -tn.
`%T2R-W-IDLETMO, ...`	Клиент ничего не присылал idletmo секунд. Нормально для клиента, который опрашивает редко — он переподключится; чтобы держать соединение, увеличьте idletmo.
`%T2R-E-LSNRAWSHR, ...`	RAW-слушатель хочет последовательный порт, который занят другим слушателем. Дайте RAW-слушателю отдельный порт или сделайте его MODBUS-слушателем.
`%T2R-W-RAWBUSY, ...`	К RAW-порту уже подключён клиент, а takeover = 0. Нормально: отключите первого клиента или поставьте takeover = 1.
`%T2R-E-TXDRAINTMO, ...`	USB-адаптер принял запрос, но не передал его в линию; запрос получает исключение 4. Адаптер или его USB-связь зависли: dmesg | tail; переподключите адаптер или перезагрузите плату. С X.00-13ECO01 линия из-за этого не блокируется.
`DEVLOCKERR / DEVBUSY на каждом запросе одного порта`	Линию держит другой запрос, который не заканчивается. Обновитесь до X.00-13ECO01 или новее (старый шлюз мог навсегда зависнуть в драйвере порта). Перед перезапуском сохраните, что делают потоки: for t in /proc/$(pidof mbusgw-t2r)/task/*; do echo $t $(cat $t/wchan); done
`%T2R-E-SESSFAIL, ... pthread_create()->11`	Слишком много TCP-клиентов сразу: каждому нужен поток, а systemd ограничивает сервис (TasksMax, на маленькой плате 578). Найдите, кто держит столько соединений (ss -tn); если все настоящие — поставьте TasksMax=2048 в разделе [Service] юнита и выполните systemctl daemon-reload.
`%T2R-E-LSNRERR, ... errno: 98`	TCP-порт занят: второй экземпляр шлюза или другая программа. Найдите: sudo ss -tlnp | grep 502.
`%T2R-E-LSNRERR, ... errno: 13`	Порты ниже 1024 требуют root. Запустите под sudo или возьмите порт выше 1024 (например 5502).
`%T2R-E-SERRANGE, ... out of range [a..b]`	Значение в настройках вне диапазона; допустимый диапазон написан прямо в сообщении. Исправьте значение.
`%T2R-E-NOSERIALS, No serials have been defined!`	Ни одна запись serials не прошла проверку. Читайте строки ошибок выше — каждая отброшенная запись объясняет причину.

Вы видите	Что это значит / что делать
`Данные неверные или сдвинутые`	Путаница порядка байт у клиента: регистры MODBUS — 16-битные слова big-endian; проверьте, как клиент собирает 32/64-битные значения.

7.2. Справочник кодов сообщений

Код	Сер.	Когда появляется
`REVISNF`	I	При старте: версия программы. Указывайте её, когда просите помощи.
`DEVREADY`	S	Последовательный порт открыт; показаны фактическая скорость, таймаут ответа и интервал t3.5.
`LSNRRDY`	S	TCP-порт слушает; показаны адрес, порт и целевое устройство.
`NETCONN`	S	Подключился TCP-клиент; показаны его адрес:порт и listener.
`NETDISCN`	S	TCP-клиент отключился; показаны его адрес:порт.
`DEVOPNERR`	E	Последовательный порт не открывается; errno объясняет почему (2 — файла нет, 13 — права, 16 — занят).
`LSNRERR`	E	Отказ bind()/listen() для TCP-порта; errno объясняет почему (98 — занят, 13 — привилегии).
`NOANSWER`	E	Устройство не ответило за таймаут iotmo.
`FRAMETMO`	W	Ответ начался, но не был завершён за таймаут.
`BADFRAME`	E	Длина принятого кадра вне разумного диапазона [5..256] октетов.
`CRC16ERR`	E	Контрольная сумма принятого кадра не сходится; показаны оба значения.
`EXCRPT`	W	TCP-клиенту возвращается MODBUS-исключение; показаны код и его имя.
`EXITST`	I	Шлюз завершается; показаны флаг выхода и итоговый статус.
`SYSERR`	E	Сбой системного вызова; показаны его имя, код возврата и errno. Приложите строку к вопросу.
`NETSYSERR`	E	Сбой системного вызова на TCP-соединении (recv, send, poll, accept); соединение закрывается.
`NETSYSWRN`	W	Не удалось установить опцию сокета; шлюз продолжает работу.
`TTYSYSERR`	E	Сбой системного вызова на последовательном порту (poll, read); запрос получает исключение 4.
`TTYSYSWRN`	W	Сбой второстепенного вызова на порту (tcflush, poll/read при очистке); шлюз продолжает работу.
`DEVSYSERR`	E	Сбой вызова настройки порта (tcgetattr, tcsetattr, cfsetspeed, mutex); порт не используется.
`DEVSYSWRN`	W	Ожидание последовательной линии затянулось (pthread_mutex_timedlock); шлюз пробует снова.
`PEERCLOSE`	I	TCP-клиент закрыл соединение — обычное окончание сеанса.
`ALLSENT`	W	На TCP-соединении нечего отправлять — безвредно.
`BADPROTO`	E	У TCP-запроса ненулевой идентификатор протокола MBAP: клиент говорит не на MODBUS TCP; соединение закрывается.
`PDUSHORT`	E	TCP-запрос слишком короткий для MODBUS; соединение закрывается.
`PDULONG`	E	TCP-запрос длиннее 256 октетов; соединение закрывается.

Код	Сер.	Когда появляется
IDLETMO	W	TCP-клиент молчал idletmo секунд (по умолчанию 60); соединение закрывается.
NETRDTMO	E	TCP-клиент начал запрос, но не закончил его вовремя; соединение закрывается.
REQFAIL	W	На запрос ответили исключением MODBUS; причина — в строках выше.
NETPOLLEV	W	TCP-соединение оборвалось: клиент сбросил его (например, перезапустили программу опроса); соединение закрывается.
SESSFAIL	E	Нет потока для нового TCP-клиента: слишком много соединений сразу (см. Траблшутинг); клиент сбрасывается.
LSNRABRT	W	TCP-порт закрыт — при остановке шлюза это нормально.
TGTOPNERR	E	TCP-порт не открыт, потому что не удалось подготовить его последовательный порт; причина — в строке выше. С X.00-13ECO09 отсутствие устройства TCP-порт не останавливает (DEVABSENT).
NOLSNRUP	E	Не открылся ни один TCP-порт; шлюз останавливается.
NOROOM	E	Ответ не помещается в буфер — внутренняя ошибка, сообщите о ней.
MBAPLEN	E	Поле длины MBAP не совпадает с принятыми данными; запрос отклоняется.
REQSHORT	E	Запрос слишком короткий, чтобы передать его в линию.
TGTNDEF	E	Слушатель ссылается на устройство, которого нет в разделе serials.
CFGNOSECT	E	В файле настроек нет раздела serials или listeners; шлюз останавливается.
CFGTRUNC	W	Слишком много записей в разделе; используются только первые.
SERBADREC	E	Запись serials повреждена или без device; пропускается.
SERBADOPT	E	В записи serials нет chars или значение слишком длинное; запись пропускается.
SERNORS485	W	Задано rs485 = 1, но эта сборка без поддержки RS485; опция игнорируется.
SERBADCHR	E	Значение chars не разобрано; пишите так: "9600, 8, N, 1".
SERRANGE	E	Скорость, биты данных или стоп-биты вне диапазона; допустимый диапазон — в сообщении.
SERBADPAR	E	Неизвестная буква чётности; используйте N, E или O.
DEVADDED	I	Запись serials принята; показаны параметры линии.
TSENABLED	I	Псевдоустройство меток времени включено для этого порта.
TSDISABLED	I	Псевдоустройство меток времени выключено для этого порта (по умолчанию).
NOSERIALS	E	Ни одна запись serials не прошла проверку; шлюз останавливается.
LSNBADREC	E	Запись listeners повреждена (нет bind, нет target, UDP); пропускается.
LSNBADBIND	E	Значение bind не разобрано; пишите так: "TCP:0.0.0.0:502".
LSNRANGE	E	Номер TCP-порта вне диапазона [1..65535].
LSNBADADDR	E	IP-адрес в bind — не правильный адрес IPv4.
LSNNOTGT	E	Устройства из target нет в разделе serials; запись пропускается.

Код	Сер.	Когда появляется
LSNADDED	I	Запись listeners принята; показаны адрес, порт, устройство, ограничения и режим (текст заканчивается на ", Mode: MODBUS", ", Mode: RAW" или ", Mode: RAW (takeover)").
NOLSNRS	E	Ни одна запись listeners не прошла проверку; шлюз останавливается.
CFGPARSE	E	Файл настроек не читается или с синтаксической ошибкой; показаны имя файла и строка.
SIGERR	E	Не удалось установить обработчик сигнала — внутренняя ошибка, сообщите о ней.
CFGERR	E	Конфигурацию не удалось обработать; причина — в строках выше.
CFGABORT	F	Шлюз останавливается из-за ошибок конфигурации выше.
DEVLOCKERR	E	Последовательная линия была занята 39 секунд (её держит другой запрос); запрос получает исключение 4.
TXNODATA	W	Пустой запрос не отправлен — безвредно.
XMITERR	E	Запрос не записался в последовательный порт; запрос получает исключение 4.
XMITSHORT	E	В последовательный порт записалась только часть запроса.
RXBUFFULL	W	Устройство прислало больше 256 октетов; остаток отброшен.
RXNODATA	E	Ответ начался, но не собрано ни одного октета.
RESPMISM	W	Ответ пришёл от другого slave или на другую функцию: два мастера или два устройства с одним адресом.
BADSPEED	W	Скорость нестандартная; используется 9600.
NOTTTY	W	Устройство — не последовательный порт.
RS485OFF	W	Режим RS485 для этого порта не включился; порт работает без него.
RS485ON	S	Режим RS485 для этого порта включён.
DEVINIERR	E	Последовательный порт не настроился; причина — в строке выше.
DEVINIT	I	Последовательный порт настроен.
DEVCLSERR	E	Последовательный порт не закрылся.
DEVNOTINI	W	Закрытие порта, который не открывался — безвредно.
DEVBUSY	E	Последовательную линию не удалось занять под запрос; запрос получает исключение 4.
CONNLIMIT	W	Новый TCP-клиент сразу отключён: у порта уже connlm клиентов.
LSNOPTRNG	E	Параметр listeners (connlm, idletmo, takeover) вне диапазона (в режиме RAW connlm может быть только 1); запись пропускается.
RAWSTART	I	RAW-сессия началась: показаны адрес клиента и последовательный порт. Делать ничего не нужно.
RAWSTOP	I	RAW-сессия закончилась; показано, сколько октетов ушло в линию и сколько пришло из линии. Делать ничего не нужно — по цифрам видно, шли ли данные вообще.
RAWKICK	W	Новый клиент занял RAW-порт (takeover = 1): старое соединение закрыто. Если это само повторяется снова и снова, две программы борются за один порт — остановите одну из них или поставьте takeover = 0.
RAWBUSY	W	Новому клиенту отказано: RAW-порт занят, а takeover = 0. Подождите, пока первый клиент уйдёт, или поставьте takeover = 1.

Код	Сер.	Когда появляется
LSNBADMODE	E	Значение mode неизвестно; запись пропускается. Пишите "MODBUS" или "RAW".
LSNRAWSHR	E	Последовательный порт RAW-слушателя занят другим слушателем; запись пропускается. Дайте RAW-слушателю свой порт.
LSNRAWOPT	E	Параметр listeners не разрешён в этом режиме (takeover в режиме MODBUS); запись пропускается. Уберите параметр или поставьте mode = "RAW".
TXDRAINTMO	E	Адаптер не передал запрос в линию вовремя; запрос сброшен и получает исключение 4.
ECHOMISM	W	Ответ на запрос записи — не его эхо (другой адрес или значение); запрос получает исключение 4.
LATEANS	W	После неудачного запроса за lategd из линии что-то пришло — опоздавший ответ; он выброшен.
DEVLOST	E	Последовательный порт пропал (выдернут USB-адаптер); он закрыт и открывается снова раз в секунду, запросы пока получают исключение 10.
DEVABSENT	W	При старте последовательный порт не открылся; TCP-порт всё равно работает, порт открывается раз в секунду.
CTLREADY	S	Канал управления для mbus-ctl открыт; показан файл-сокет.
CTLOFF	I	Канал управления выключен опцией /control=NONE; mbus-ctl до этого шлюза не достучится.
CTLBUSY	W	Этим файлом-сокетом уже пользуется другой работающий шлюз; этот работает без канала управления. Дайте каждому шлюзу свой /control=<файл>.
CTLERR	W	Файл-сокет не создаётся (2 — нет каталога, 13 — нет прав, 88 — там не сокет); шлюз работает без канала управления.
CTLSYSWRN	W	Системный вызов канала управления не удался; шлюз работает дальше.
CTLRESET	I	Оператор отключил этого клиента (mbus-ctl reset connection). Ничего делать не нужно.
CTLRESTART	I	Оператор попросил перезапуск, файл настроек хороший; шлюз перезапускается.
CTLCHKFAIL	E	restart gateway отклонён: в файле настроек ошибки (mbus-ctl их показывает). Шлюз работает дальше. Исправьте файл и повторите.
CHECKOK	S	/check: файл настроек хороший.
RESTARTING	I	Шлюз запускает сам себя заново с той же командной строкой; дальше обычные строки запуска.
EXECERR	F	Перезапуск не удался; шлюз останавливается. systemd и procd запустят его снова; вручную — запустите снова.
CTLNOGW	E	mbus-ctl не достучался до шлюза. errno 2: не запущен или другой файл-сокет (/socket=); 13: нет прав — sudo; 111: не запущен, остался старый файл.
CTLANSERR	E	Шлюз отклонил запрос; показана его причина.
CTLBADANS	E	mbus-ctl не понял ответ — шлюз другой версии? Приведите строку.
CTLWAIT	I	mbus-ctl ждёт перезапущенный шлюз.
CTLBACK	S	Шлюз вернулся после перезапуска.
CTLNOBACK	W	Шлюз не ответил после перезапуска вовремя; читайте его журнал.

Код	Сер.	Когда появляется
CTLRESETN	S	mbus-ctl reset connection отключил столько клиентов.

7.3. Если ничего не помогло

Соберите и приложите к вопросу: 1) полный журнал запуска с /trace (от REVISNF до первой ошибки); 2) ваш файл настроек; 3) вывод `ls -l <устройство>` и `dmesg | tail -20`; 4) точную модель RTU-устройства и его паспортные параметры линии. С этими четырьмя вещами проблема почти всегда видна с первого взгляда.